Testing

07: Programmable DMA

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Let’s talk P2

- New Version: V4
- Several bugs fixed:
  - IP Caching bug
  - Disappearing Init files (linked to IP Caching)
  - hw_platform_0 vs hw_platform_1

DMA: Direct Memory Access

- A mini-CPU that does copy for you:

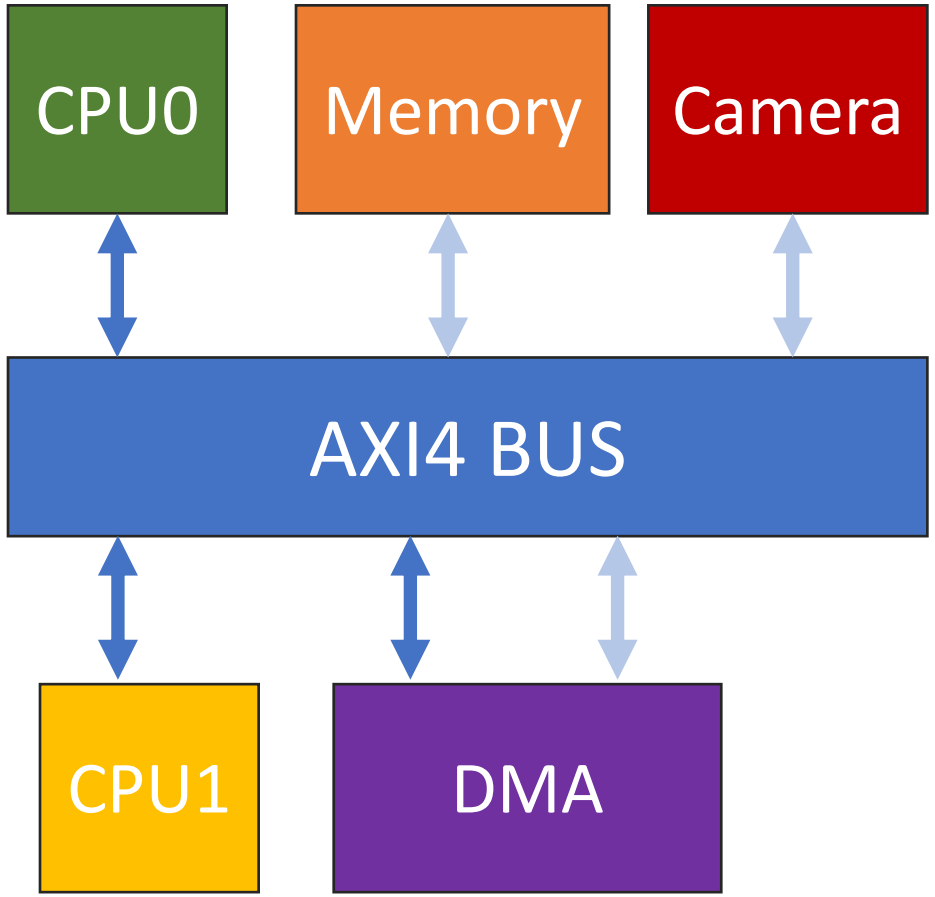
```
void copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

DMA has 2 Memory Interfaces



- Interface 1: Memory Copy
  - Data Interface
  - Fast
  - Master
- Interface 2: Tell DMA <u>what</u> to copy
  - Control Interface
  - Slower
  - Slave

DMA V1.0 Interface

- 0x0400: Control Register (Start)
- 0x0404: Status Register (Done)
- 0x0408: Source Address
- 0x040C: Destination Address
- 0x0410: Transfer Size in Bytes

Using DMA from the CPU:

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

```
void dma_copy ( uint32_t * from,
                uint32_t * to,
                uint32_t size){

    *((uint32_t*)(0x0408))=from;
    *((uint32_t*)(0x040C))=to;
    *((uint32_t*)(0x0410))=size;
    *((uint32_t*)(0x0400))= 0x1; //start
    //spin until copy done?
    while( *((uint32_t*)(0x0404)) != 0x1){;}
}
```

`volatile` keyword omitted!

Other DMA tweaks

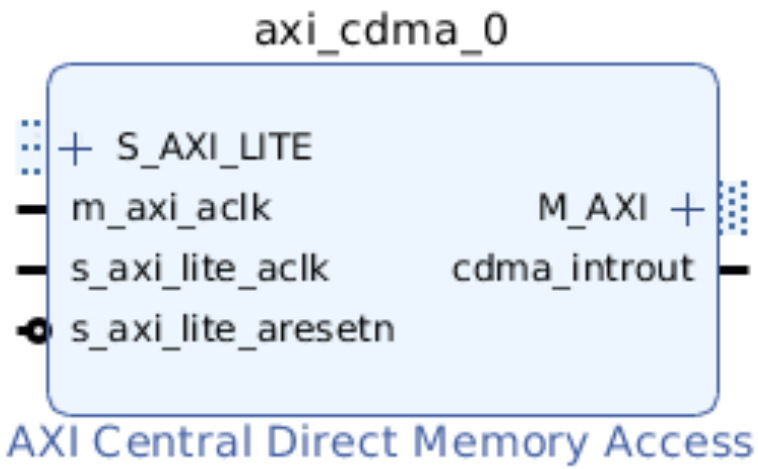
```
void dma_copy (uint32_t * from,
               uint32_t * to,
               uint32_t size,
               uint32_t inc_from, uint32_t inc_to)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = (inc_from ? *from[i] : *from);

        if (inc_to) to[i] = reg;
        else        to = reg;
    }
}
```

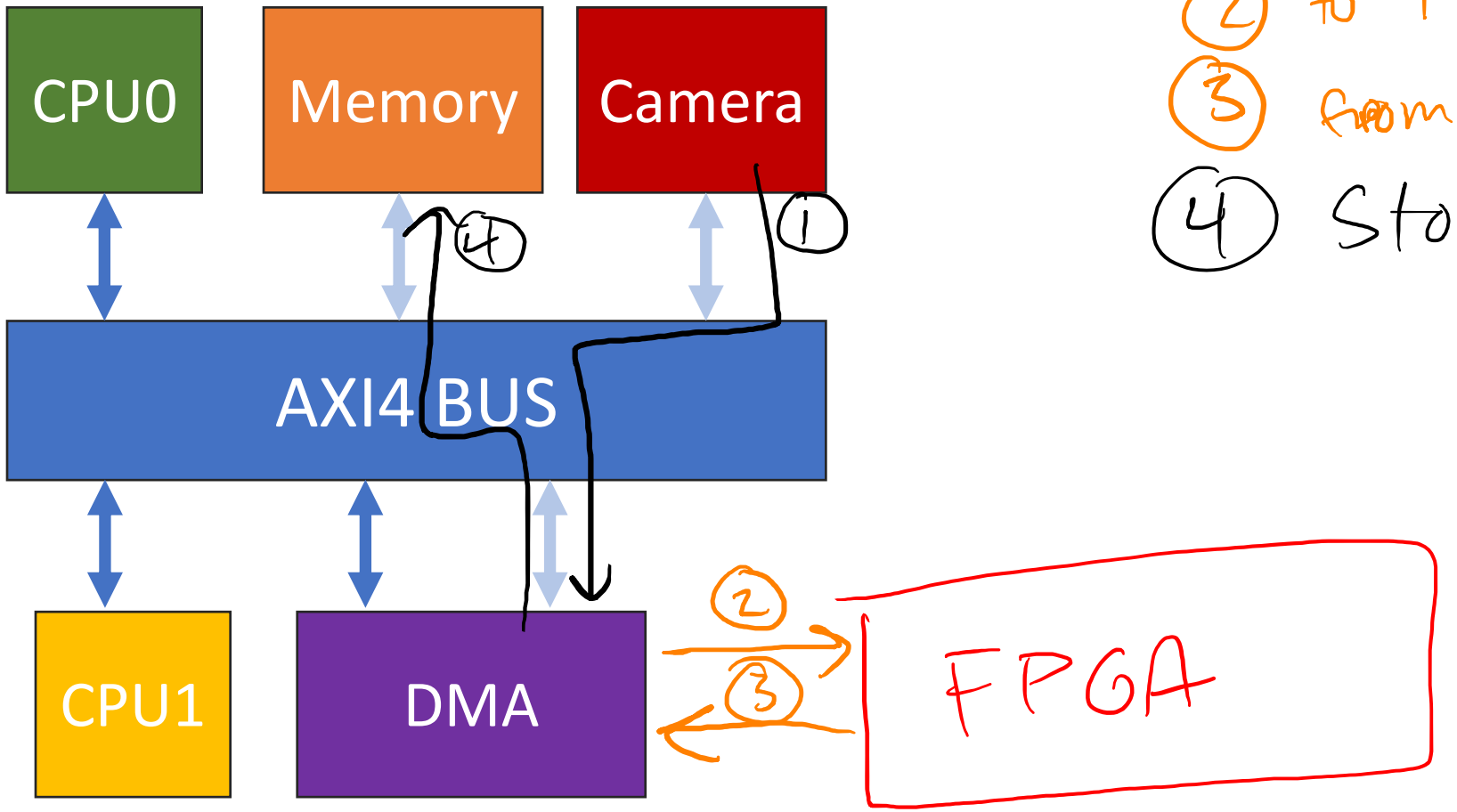
DMA in Xilinx



Xilinx terminology:

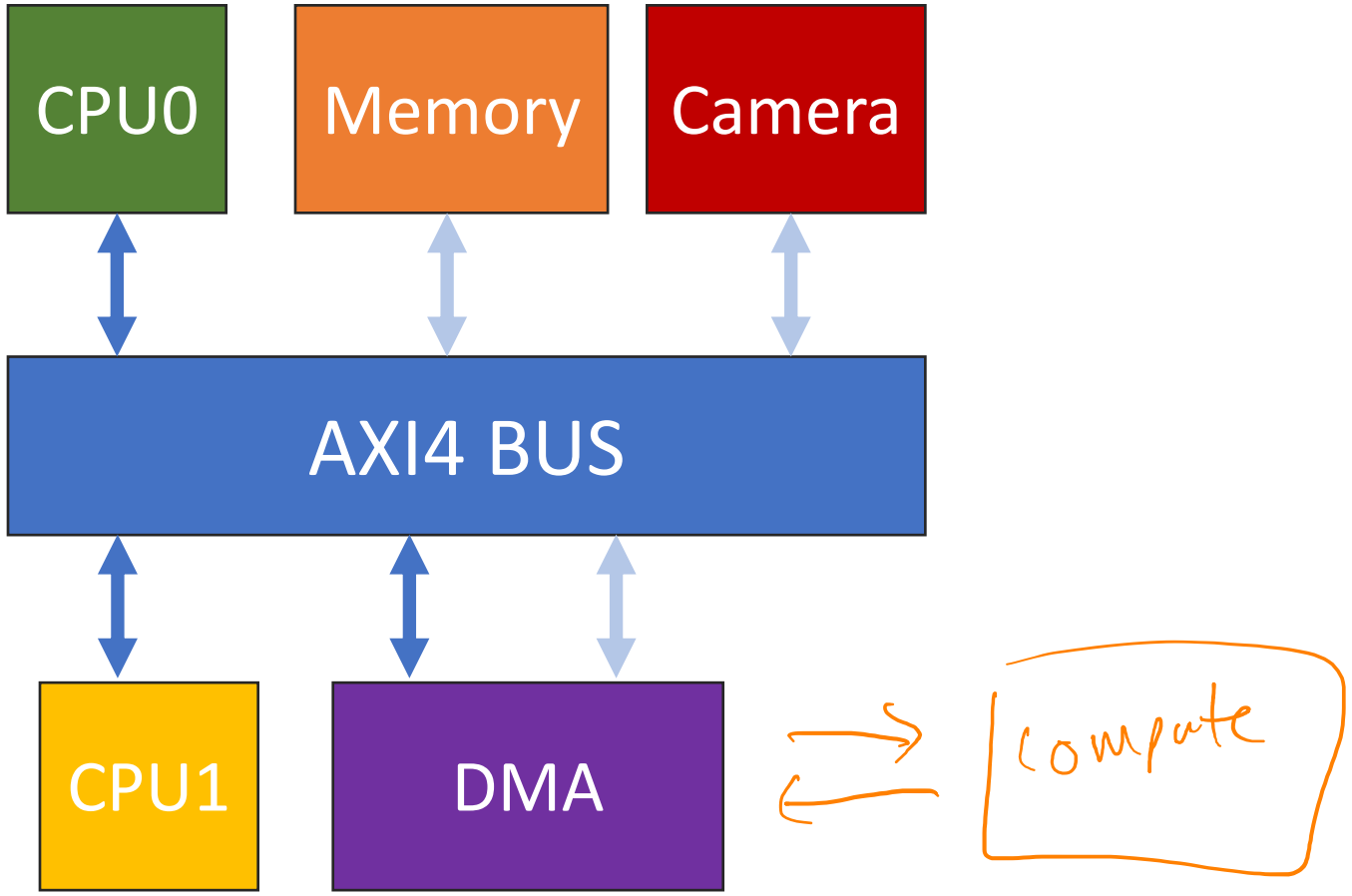
- Central Direct Memory Access : CMDA
  - Standard DMA
- Direct Memory Access (DMA)
  - Programmable DMA!

DMAs copy by load + store

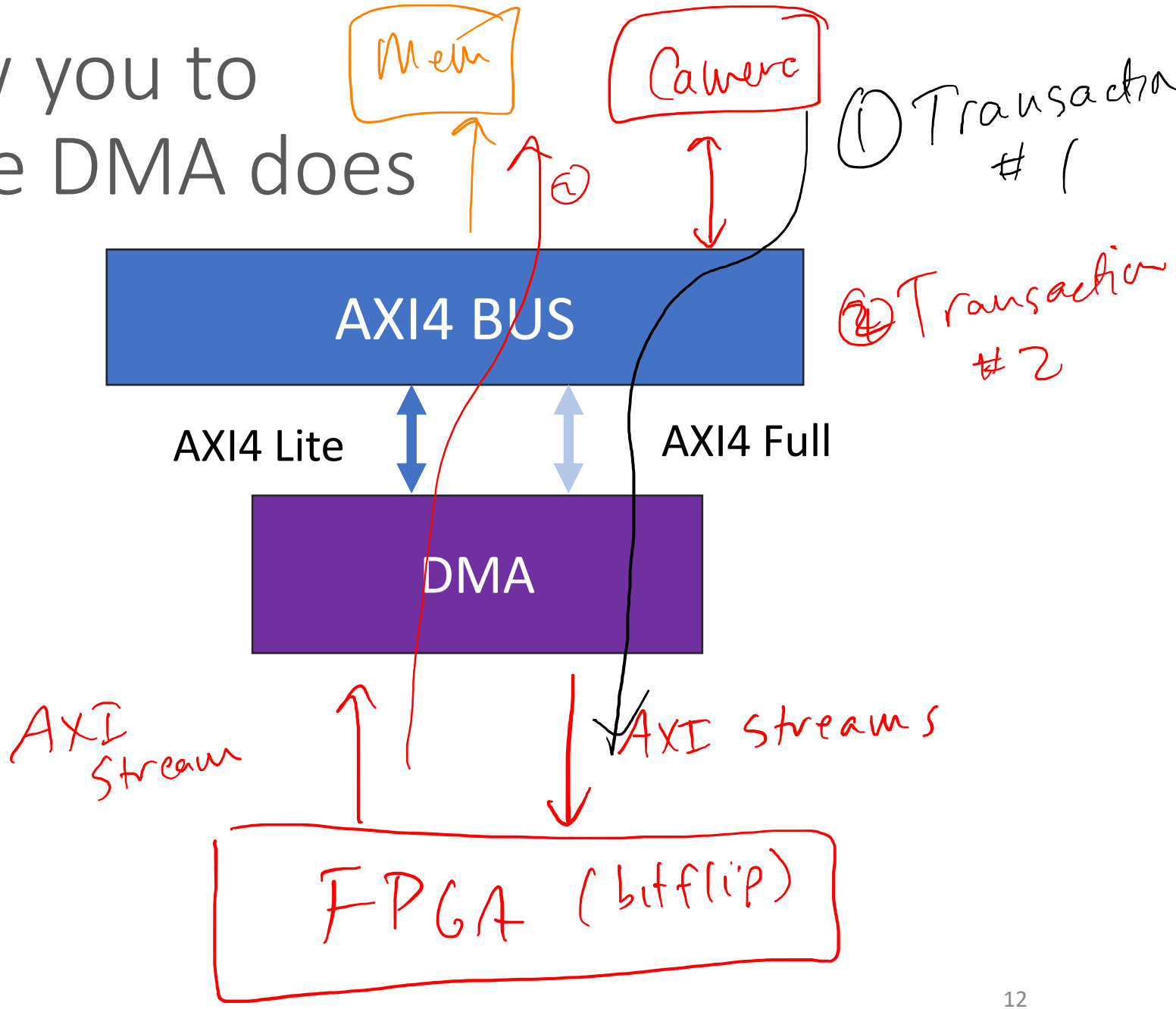


1. Load
2. to FPGA — AXI stream
3. from FPGA — AXI streams
4. Store

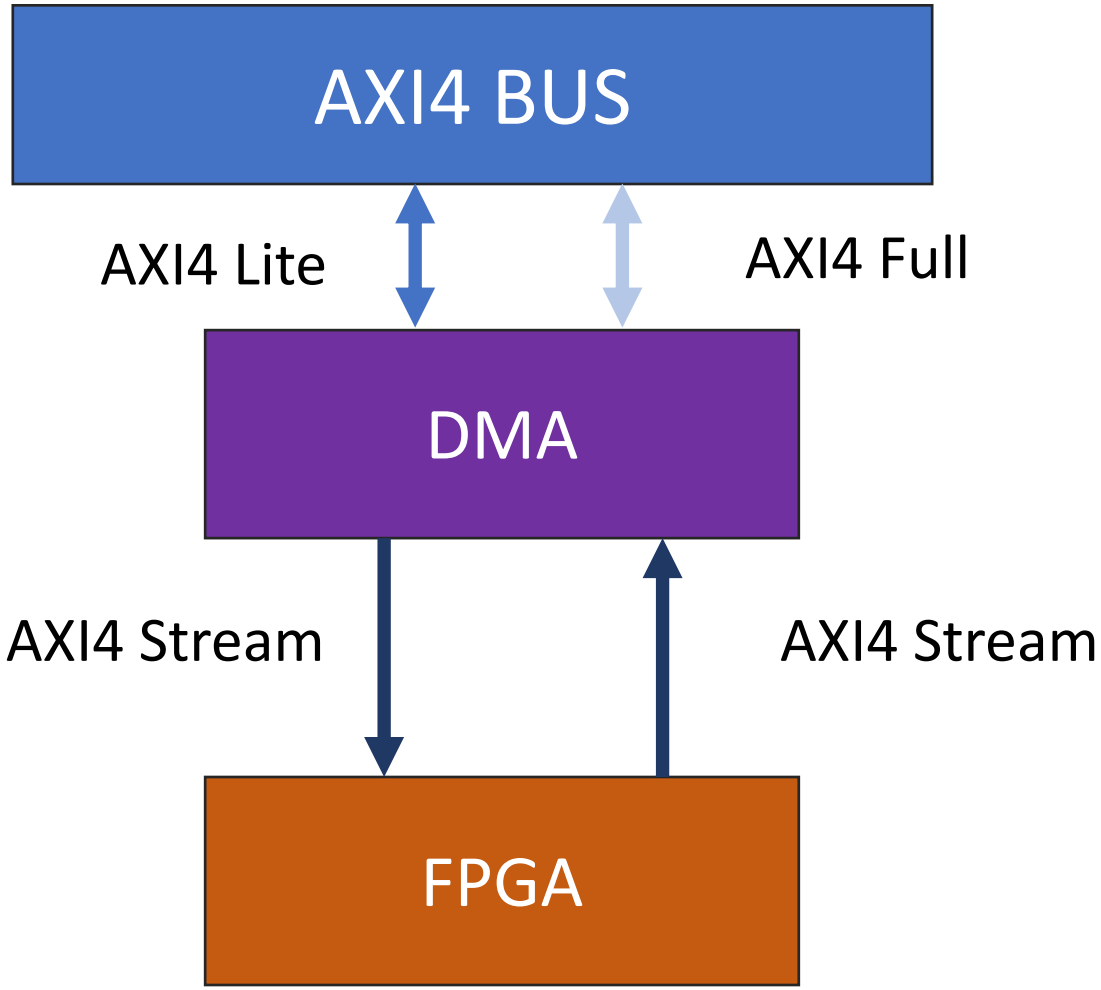
With FPGAs, DMAs can copy with load + **compute** + store!



Xilinx DMAs allow you to **program** what the DMA does

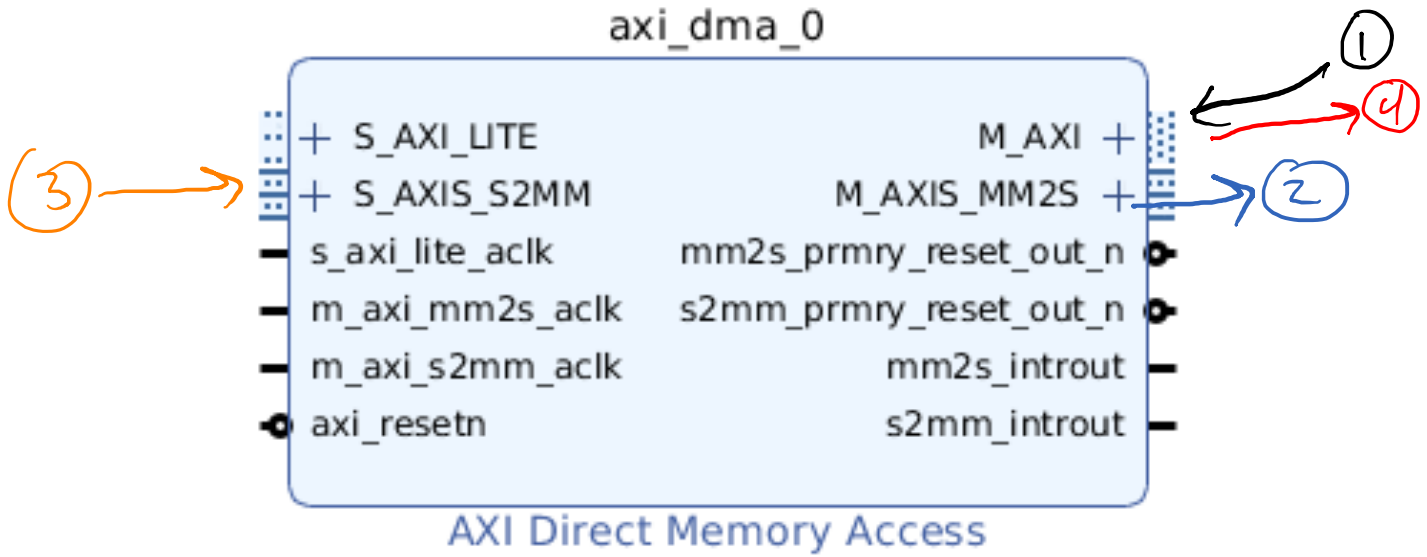


Xilinx DMAs allow you to **program** what the DMA does



Xilinx Programmable DMA

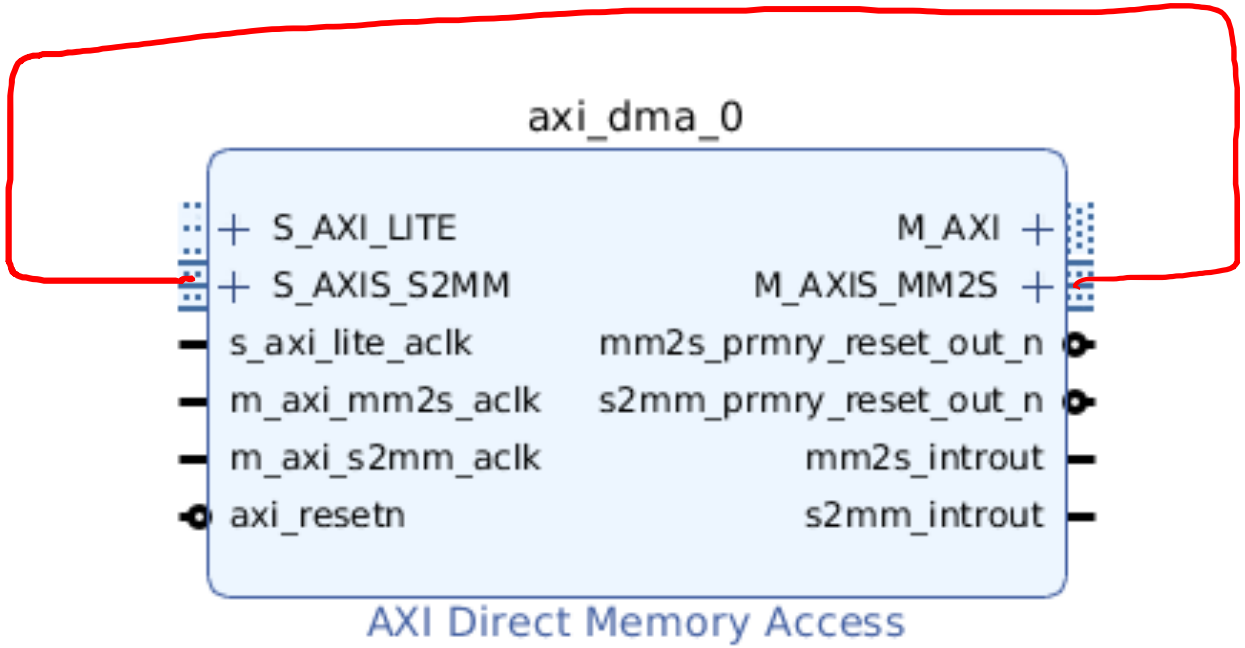
(1) Load
(2) out over stream M_AXIS_mm2s



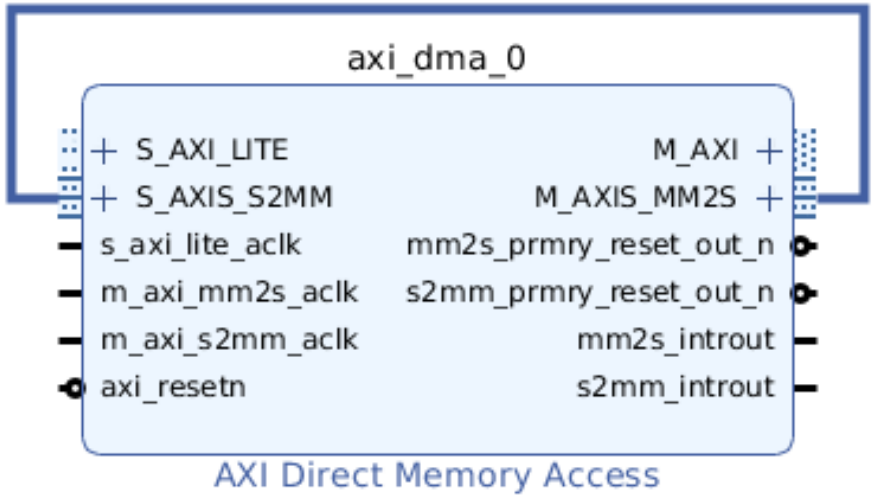
(3) in over stream

(4) Store

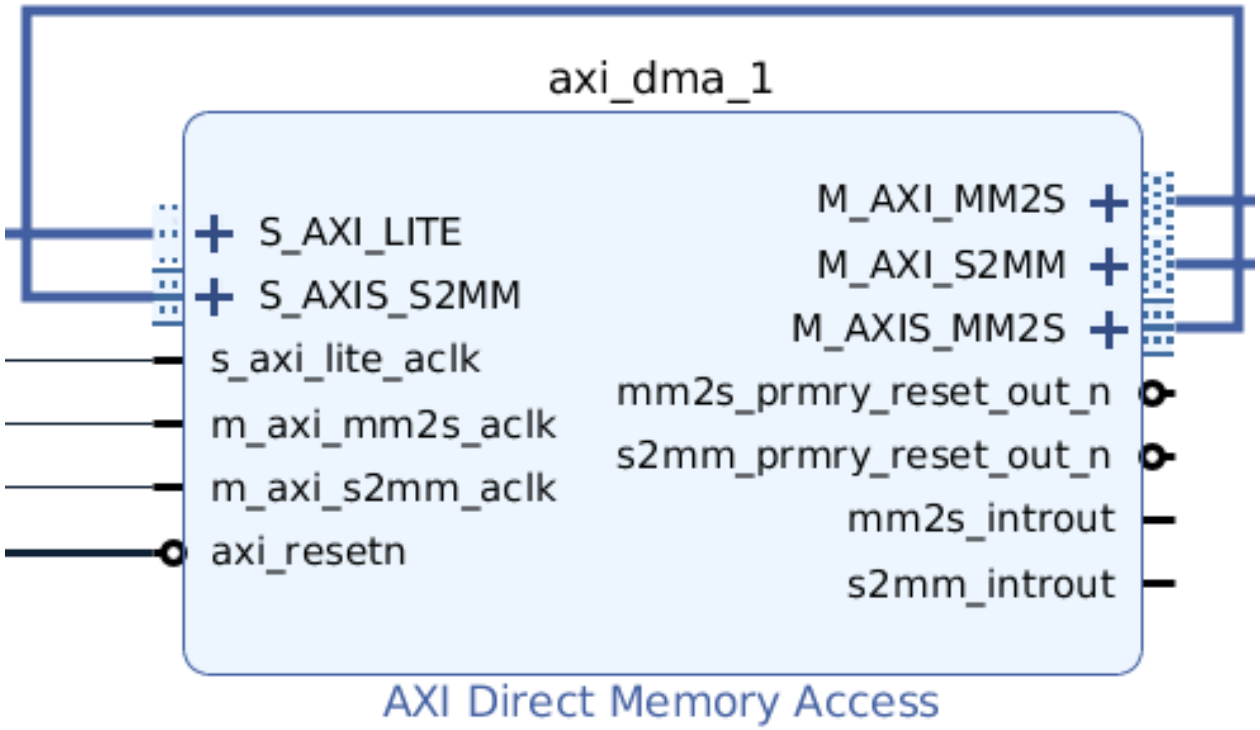
Can we make this into a regular DMA?



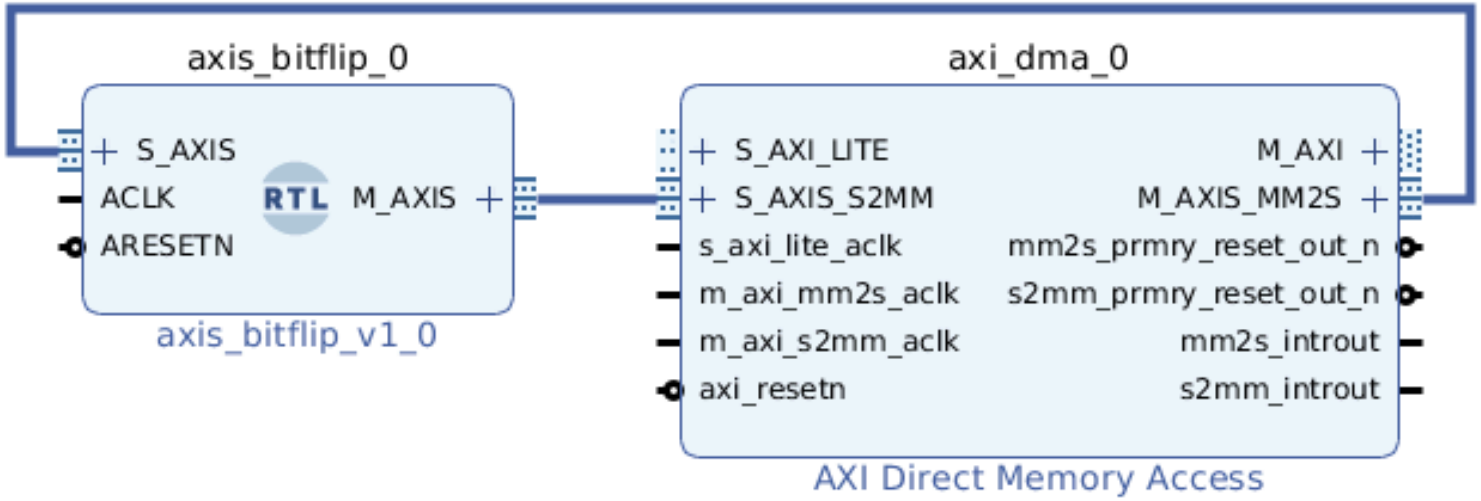
Can we make this into a regular DMA?



Xilinx DMA acting as a CDMA



DMA that flips all the bits in the copy



```
module axis_bitflip(
    input           ACLK,
    input           ARESETN,
    input [31:0]    S_AXIS_TDATA,
    input [3:0]     S_AXIS_TKEEP,
    input           S_AXIS_TLAST,
    input           S_AXIS_TVALID,
    output          S_AXIS_TREADY,
    output [31:0]   M_AXIS_TDATA,
    output [3:0]    M_AXIS_TKEEP,
    output          M_AXIS_TLAST,
    output          M_AXIS_TVALID,
    input           M_AXIS_TREADY
    );
```

```
    assign M_AXIS_TDATA= ~S_AXIS_TDATA;

    assign M_AXIS_TKEEP= S_AXIS_TKEEP;
    assign M_AXIS_TLAST = S_AXIS_TLAST;
    assign M_AXIS_TVALID= S_AXIS_TVALID;
    assign S_AXIS_TREADY= M_AXIS_TREADY;

endmodule
```

Bitflip in C

- INPUT = { 0x0, 0x1, 0x2, 0x3}
- DMA (w/bitflip)
- OUTPUT = { 0xf, 0xe, 0xd, 0xc}

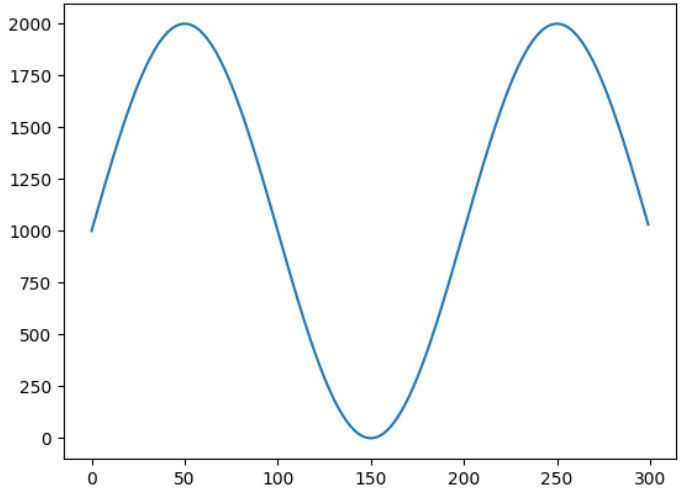
Project 3: Exponential Moving Average

- Filter signal

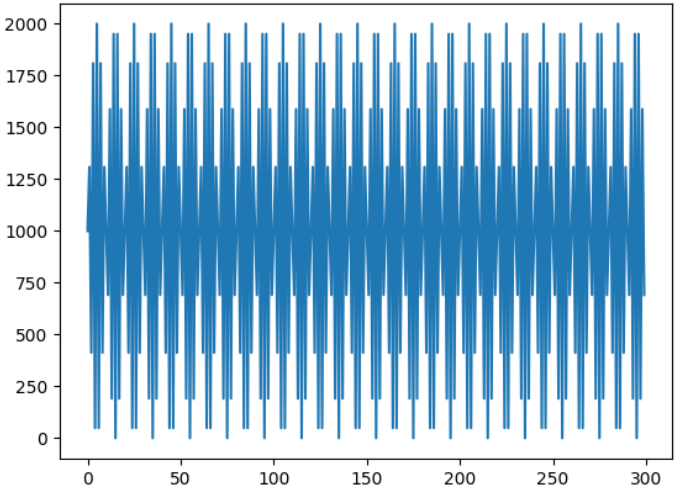
- Keep: low-frequency signal
- Reject: high-frequency signal

- How?
  - Infinite-Impulse Response (IIR) Filter

Filtering Review



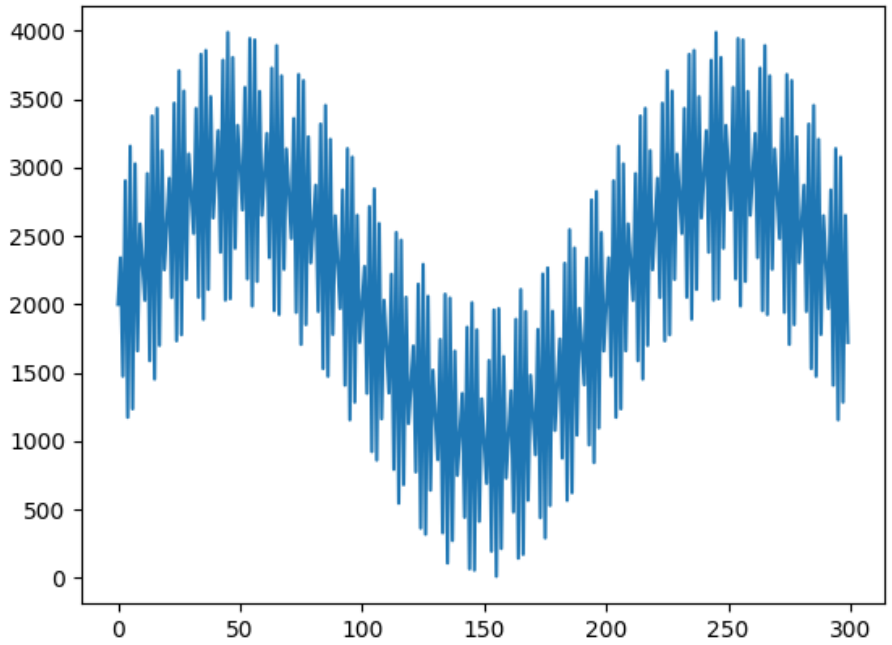
+



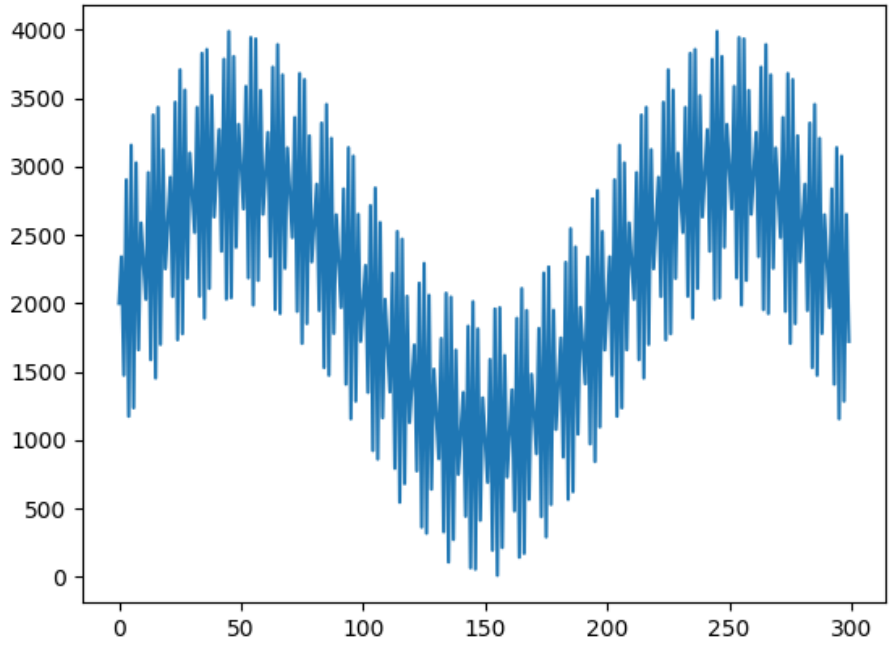
=

Filtering Review

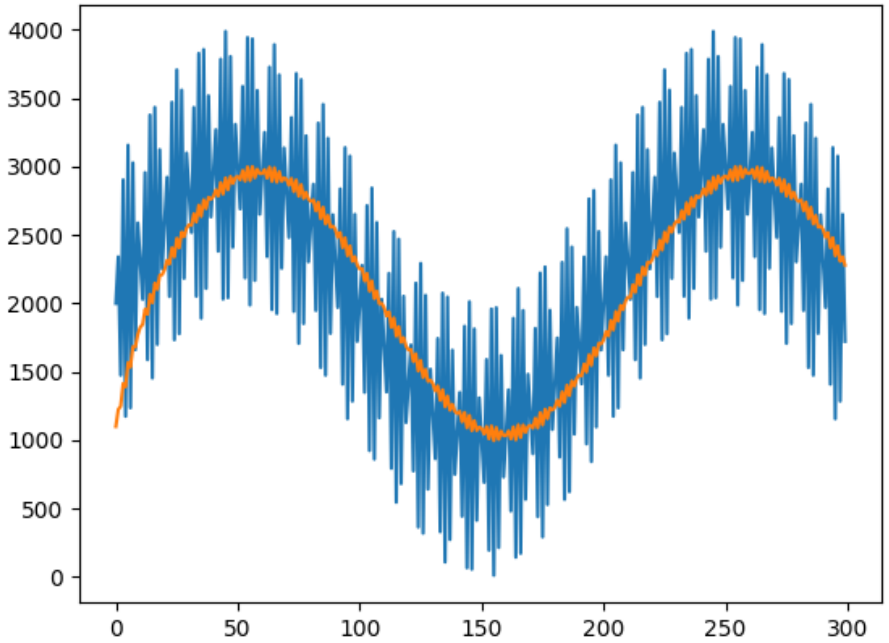
=



Filtering Review



->



Exponential Moving Average in Python

$y_{-1} = 0$

$ema(x_0) = y_0 = 0.1x_0 + 0.9(0)$

$ema(x_1) = y_1 = 0.1x_1 + 0.9y_0$

$= 0.1x_1 + 0.9(0.1x_0 + 0)$

$ema(x_2) = y_2 = 0.1x_2 + 0.9(y_1)$

$= y_2 = 0.1x_2 + 0.9(0.1x_1 + 0.9(0.1x_0))$

```
y = 1000
def ema(x):
    global y
    alpha = 0.1
    y = x * alpha + y * (1.0 - alpha)
    return y

xs = [ 1000, 1100, 1000, 900, 1000]
ys = [ ema(x) for x in xs]
```

EMA in Python vs. Verilog

Next Time

- Memory Protection

07: Programmable DMA

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University